

MODULE- 2: PROCESS MANAGEMENT

Topic: PROCESS CONCEPT

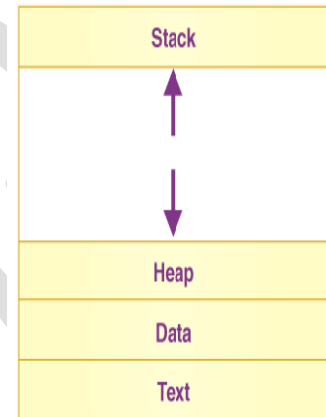
What is a **Process in an Operating System**?

- In an operating system, a process is an **instance of a program in execution**.
- It is a basic unit of work that can be scheduled and executed by the operating system.

Components of a Process

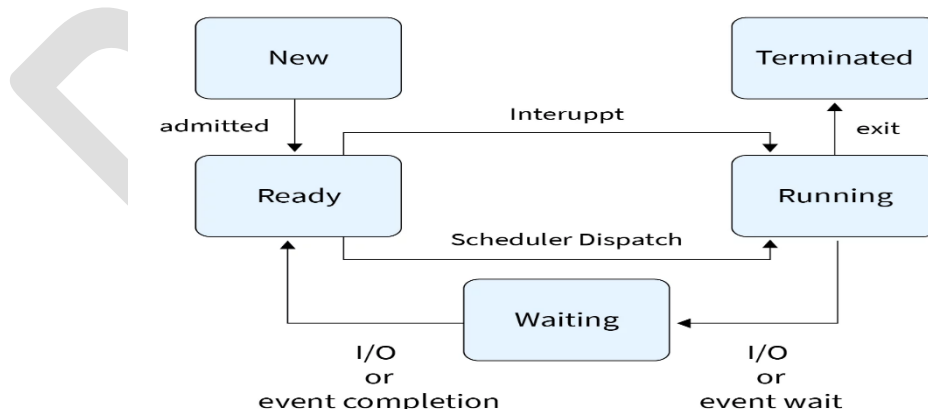
It is divided into the following four sections:

1. Program code/Text: The instructions that the process will execute.
Ex: `printf("Hello")`
2. Data: The data that the process will use during its execution.
 1. Initialized Data = `int x = 10; &`
 2. Uninitialized Data = `int count;`
3. Stack: A data structure that is used to
 - Store temporary data,
 - Function parameters and return addresses.
4. Heap: A data structure that is used to store dynamically allocated memory.



Process States:

The process life cycle is the sequence of states that a process goes through during its existence. The process life cycle can be divided into the following stages:



1. New: The process has been created but has not yet been scheduled for execution.
Example: When you double-click MS Word → the process is in New state.
2. Ready: The process is ready to be executed.
It is waiting for the CPU to be assigned.
3. Running: The process is currently being executed.

Example: When MS Word is actively responding (typing, saving), it is running.

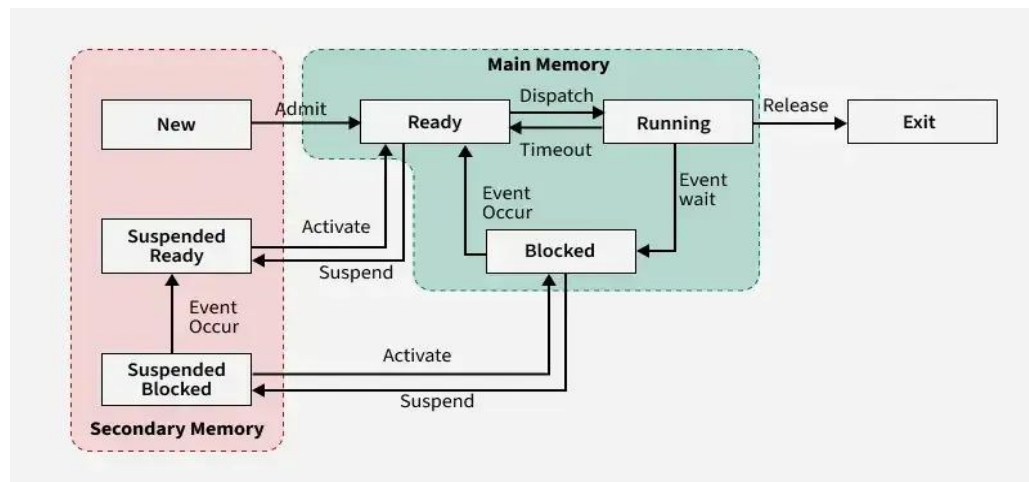
4. **Waiting:** The process is waiting for an event to occur, such as the completion of an I/O operation.

A program waiting for the user to select a file to open.

It pauses → OS moves it to Waiting state.

5. **Terminated:** The process has finished executing.

Example: You close MS Word → it enters Terminated state



Process Control Block:

- A Process Control Block (PCB) is a data structure used by the Operating System to store all information about a process.
- Without PCB, the OS cannot track or manage the process.

Why PCB is Needed?

When the CPU switches from one process to another (context switching), the OS must save:

- What the process was doing
- Where it stopped
- What resources it is using.
- All this information is stored in the PCB.

Contents of PCB:

1. **Process ID:** An identifier that helps us in identifying and locating a process.

Example: In Linux, each running program has a unique PID.

2. **Process state:** It identifies the state that the process is currently in.

[New process, ready, running, waiting or terminated]

3. **Program counter:** It holds the address of the next instruction to be executed for the process.

It also stores a count of the number of instructions in the process.

Example: If the process paused at line 45, the program counter = 45.

4. **CPU registers:** The content of processor registers gets stored here when the process is in running state. Ex: accumulators, index and general-purpose registers, instruction register and condition code register.
5. **Accounting information:** Contains details like CPU utilization, real-time used by the process, number of jobs or processes, etc.
6. **Memory-management information:** Contains value of base and limit registers, details about page and segment tables. It depends on the memory system of the OS in use.
7. **I/O status information:** Comprises I/O related information including list of I/O devices allocated to the process, status, etc.
8. **CPU scheduling information:** A process needs to be scheduled for execution. Based on this scheduling, it goes from ready to running.

Difference between Process and the Program

Feature	Process	Program
Definition	An instance of a program in execution.	A set of instructions that can be executed by a computer.
Creation	Created when a program is executed.	Created when a program is written or compiled.
Resources	Requires resources such as CPU time, memory, and I/O devices.	Does not require any resources.
State	Can be in one of several states, such as new, runnable, running, blocked, or terminated.	Always in the same state.
Control block	Has a process control block (PCB) that contains information about the process.	Does not have a control block.
Execution	Can be executed concurrently with other processes.	Can only be executed one at a time.
Example	A web browser, a word processor, a game.	A source code file, an executable file.

Topic: PROCESS SCHEDULING:

- Process scheduling is the act of deciding which process should be allocated the CPU for execution.
- A scheduler is a program that uses a scheduling algorithm to make choices.
- The scheduler selects a process from the ready queue and moves it to the running state.
- The scheduler then gives the CPU to the running process.

Because CPU is the most important and limited resource, the OS must schedule processes efficiently to get:

- Better CPU utilization
- Faster execution
- Less waiting
- Good user experience

Key Components of Process Scheduling

1□. Scheduling Queue: All processes waiting for CPU are placed in the ready queue.

2□. CPU Scheduler: Picks a process from the ready queue.

3□. Dispatcher: Gives CPU control to the selected process.

Performs context switching.

4□. Context Switch: Saving and loading process states when switching between processes.

Scheduler Criteria:

The scheduler must consider a number of factors when making a scheduling decision. These factors include:

- Priority: The priority of a process is a value that determines the order in which processes are scheduled for execution.
- CPU burst time: The CPU burst time of a process is the amount of time that the process needs to execute before it is blocked.
- I/O burst time: The I/O burst time of a process is the amount of time that the process spends waiting for I/O operations to complete.
- Arrival time: The arrival time of a process is the time at which the process is created.
- Completion Time: The time at which the process completes its execution.
- Turn Around Time: Time Difference between completion time and arrival time.

Scheduling Algorithm:

1. First-come, first-served (FCFS): In FCFS, processes are scheduled in the order in which they arrive.
2. Shortest job first (SJF): In SJF, the process with the shortest CPU burst time is scheduled next.
3. Priority scheduling: In priority scheduling, processes are scheduled according to their priority.

4. Round-robin (RR): In RR, each process is given a time slice. The processes are then scheduled in a round-robin fashion, with each process being given a time slice in turn.

Process Scheduler

- Process Schedulers are fundamental components of operating systems responsible for deciding the order in which processes are executed by the CPU.
- In simpler terms, they manage how the CPU allocates its time among multiple tasks or processes that are competing for its attention.

Types of Process Schedulers

There are three types of process schedulers:

1. Long-Term Scheduler (Job Scheduler)

The Long-Term Scheduler is responsible for loading processes from disk into main memory so they can begin execution. When a new process is admitted, it moves into the Ready state.

Key Functions

- Transfers processes from the Job Queue to the Ready Queue.
- Controls the degree of Multi-programming
- In many modern time-sharing systems (such as Windows), a long-term scheduler may not exist; new processes are directly admitted to memory for short-term scheduling.

2. Short-Term Scheduler (CPU Scheduler)

The Short-Term Scheduler (STS) is responsible for selecting a process from the ready queue and assigning the CPU to it.

Key Functions

- Frequently selects the next process to execute from the ready state.
- Uses various CPU scheduling algorithms to decide process order.
- Maximizes CPU utilization by keeping the processor as busy as possible.

Dispatcher

- The Dispatcher is a special program that takes over once the short-term scheduler selects a process. It transfers control of the CPU to the chosen process.

Functions of the Dispatcher

- Context Switching: Saves the state of the previously running process and restores the state of the new one.
- Mode Switching: Ensures correct transition into user mode from kernel mode.
- Program Control Transfer: Jumps to the correct starting point in the newly selected program.

Dispatcher Example

Using FCFS scheduling for processes $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$:

- Scheduler selects P1 first.
- Dispatcher loads P1 onto the CPU.
- Next the scheduler selects P2, and the dispatcher assigns P2 to the CPU, and so on.

3. Medium-Term Scheduler

The Medium-Term Scheduler (MTS) manages swapping, which temporarily moves processes between main memory and disk.

Key Functions

- Swaps processes out of memory when they are waiting (e.g., blocked for I/O) to reduce the degree of multiprogramming.
- Frees memory for other active processes.
- Swaps processes back into memory when they are ready to continue execution, allowing them to resume from where they left off.
- Helps maintain an effective mix of CPU-bound and I/O-bound processes.

Scheduler	Function	Works When	Example
Long-term	Admits processes to ready queue	Start of process	College admissions
Short-term	Selects process for CPU	Runs very frequently	Teacher calling next student
Medium-term	Swaps processes in/out of memory	System overloaded	Sending students out and calling back

Scheduling Queues:

1. Job Queue:

- It consists of all processes in the system.
- As processes enter the system, they are put into this queue.
- The job queue contains all processes that enter the system.
- These are processes that are submitted for execution but may not be in memory yet.

Example

Imagine 10 students joining a lab queue to use the computer.

All 10 students = job queue

2. Ready Queue:

- It consists of processes which are residing in main memory, ready to run and waiting for CPU.

- All the processes in “Ready” state are put in this Queue.

Example

Think of students sitting in front of the computer waiting for their turn to type.

They are ready and only waiting for CPU time.

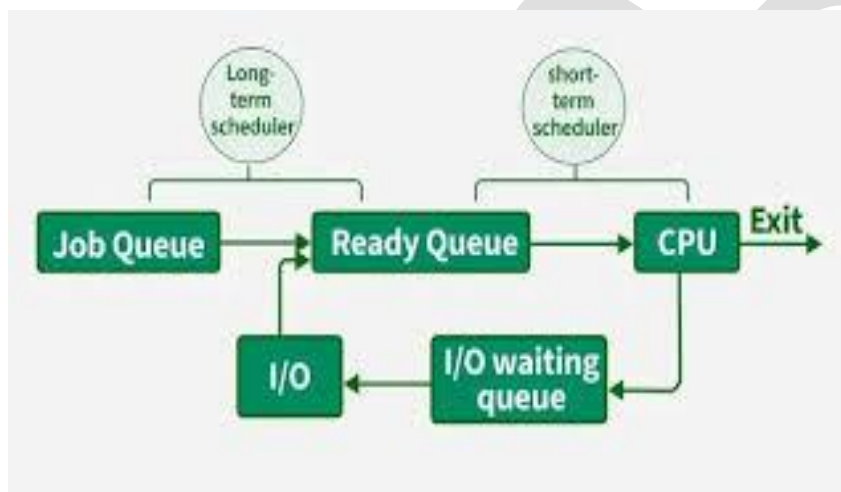
3. Device Queue:

- It consists of processes which are waiting for a particular device(resource).
- Each device has its own device queue

Example

If 3 processes want to read from disk, they wait in the disk device queue.

Like students waiting for the printer to become free = printer queue.



Scheduling Algorithms

These are the main CPU scheduling algorithms you must teach:

- 1□.FCFS (First Come First Served)
- 2□.SJF (Shortest Job First) – non-preemptive
- 3□.Preemptive SJF / SRTF (Shortest Remaining Time First)
- 4□.Priority Scheduling – preemptive & non-preemptive
- 5□.Round Robin (RR)
- 6□.Multilevel Queue Scheduling
- 7□.Multilevel Feedback Queue Scheduling

Topic: OPERATIONS ON PROCESSES

Process: A program that is under execution is known as a process.

Operation on the process:

The user can perform the following operations on a process in the operating system:

1. Process creation
2. Process scheduling or dispatching
3. Blocking
4. Pre-emption
5. Termination

1. Process Creation: Process creation is the initial step to process execution. It implies the creation of a new process for execution.

When are new processes created?

- New program loaded (e.g., opening Chrome)
- New task started (e.g., starting a download)
- System calls like fork(), exec()
- User requests (clicking an app icon: Facebook)

2. Process scheduling\dispatching: Scheduling refers to the event where the OS puts the process from ready to running state.

3. Process Termination: Ending a process is known as process termination.

There are many events that may lead to process termination, some of them are:

One process terminating the other process.

Why processes terminate?

- Normal completion (exit())
- Error occurred (divide by zero)
- Parent aborts child
- Child exceeds resource limit
- User manually closes (Task Manager → End Task)

4. Process Suspension:

- Process suspension takes place when the OS moves a process out of the main memory (RAM) and temporarily places it on disk.(RAM -> Secondary Memory)
- It usually happens when a process enters the blocked state.

Two ways:

- Blocked_Suspend – The process is blocked and also not in memory.
- Ready_Suspend – The process is ready but hasn't been loaded into memory yet.

5. Process Resuspension:

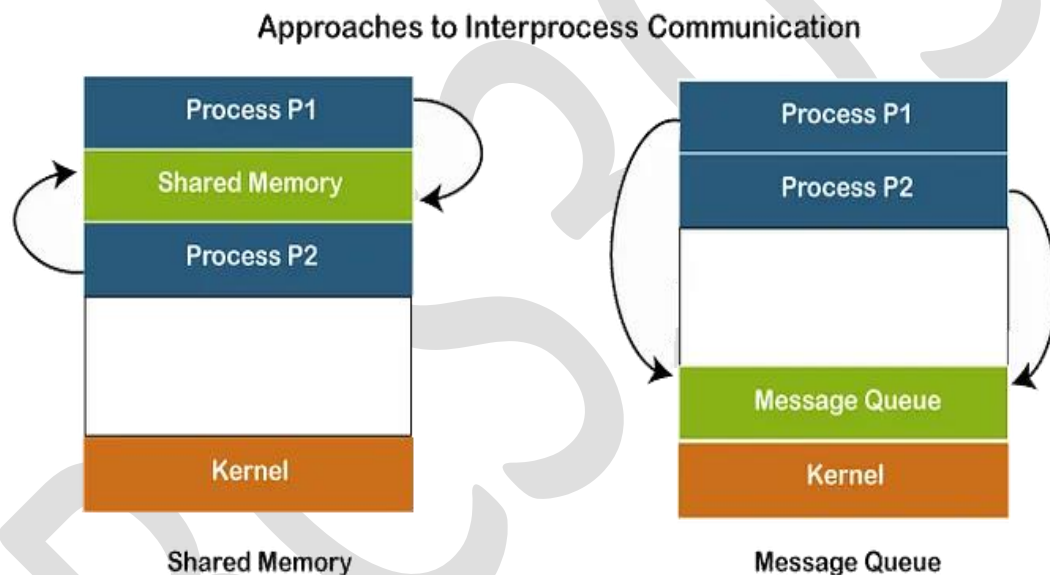
- Process resuspension is the act of moving a previously suspended process back into main memory and making it available for execution.

Topic: INTER-PROCESS COMMUNICATION (IPC)

- IPC allows two or more processes to exchange data and coordinate their actions.
- IPC mechanisms enable processes to work together, share resources, and coordinate operations to achieve a common goal

Two Types of IPC:

1. **Shared Memory**
2. **Message Passing**



1□.Shared Memory

Two processes share a common memory region.

How it works:

- A shared memory segment is created.
- Both processes read/write directly into it.
- Very fast (no OS involvement after setup).
- Since both processes access the same memory segment, this method is fast Example:
- Process A writes: $x = 10$ into shared memory.
- Process B reads $x = 10$ from the same shared memory.

1. Message Passing:

- Message Passing is a method where processes communicate by sending and receiving messages to exchange data.
- Processes send and receive messages through OS-managed mailboxes or message queues.

How it works:

- OS provides send() and receive() functions.
- Messages can be short or long.
- No shared space: communication happens through kernel.

Example:

- Process A → send (Temperature = 40°C)
- Process B → receive (Temperature = 40°C)

Good for:

- Distributed systems
- When processes don't share memory

Topic: Multi-threading Programming**Thread:**

- Thread is a basic unit of execution in process.
- Each process can have a number of threads executing in it.
- A thread is also called a lightweight process.
- Each process contains: Process resources + one or more threads

Components of Thread:

- A thread ID
- A program counter
- A register set
- A stack

Example:

MS Word:

- One thread for typing
- One thread for spell-check
- One thread for autosave

Chrome:

- One thread for Opening website
- One thread for Downloading
- One thread for search options.

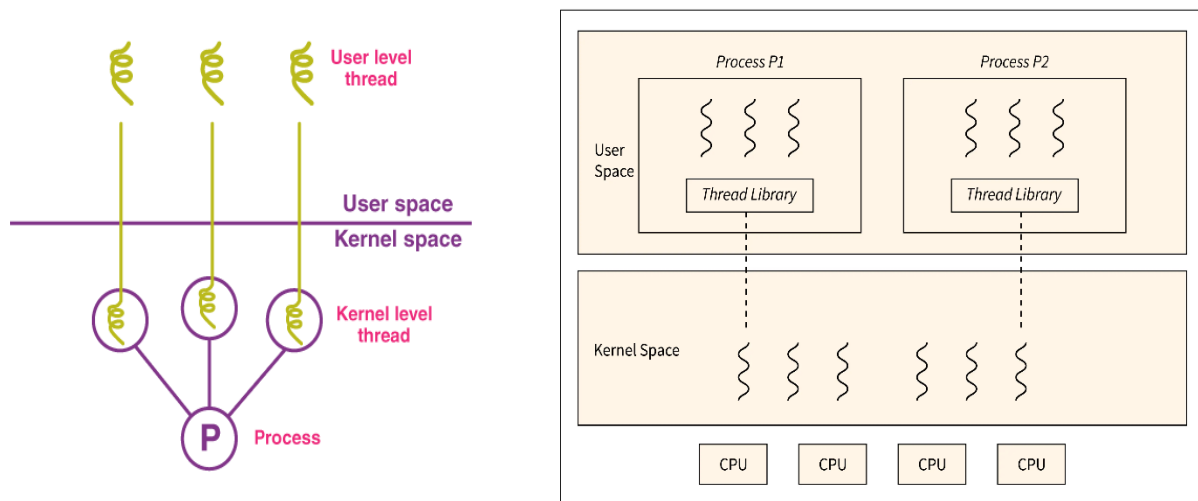


Fig: User- Kernel Level thread

Types of Thread:

Threads are implemented in following two ways:

- User Level Threads – User managed threads.
- Kernel Level Threads – Operating System managed threads acting on kernel, an operating system core.

1. User Level Thread (ULT) :

- User Level Thread is implemented in the user level library.
- These are created in user level space
- They are not created using the system calls.
- Kernel doesn't know about the user level thread

Example: Java, POSIX, and other languages

Advantages: Simple to create since no intervention of kernel.

Disadvantages: If one thread causes a page fault, the entire process blocks.

2. Kernel Level Thread (KLT):

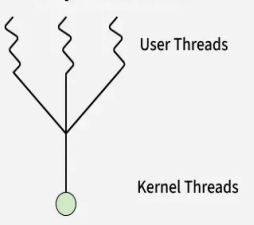
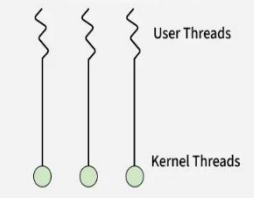
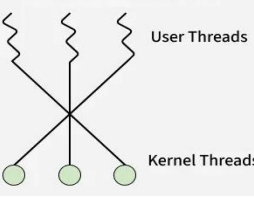
- Kernel knows and manages the threads.
- These are created in kernel level space
- Instead of thread table in each process, the kernel itself has thread table (a master one) that keeps track of all the threads in the system.
- Advantages: Since kernel has full knowledge about the threads in the system,
- Disadvantages: Slow and inefficient

Multithreading:

- Multithreading is a technique where a process is divided into smaller execution units called threads that run concurrently.
- Multithreading improves system performance and responsiveness by allowing multiple threads to share CPU, memory and I/O resources.
- Example: In a browser, each tab can be a thread.
 - In MS Word: one thread formats text while another processes inputs.
 - Instagram: one thread for login, one thread for scroll,, one for videos..etc

Multithreading model are of three types.

1. Many to many model.
2. Many to one model.
3. One to one model.

1. <u>Many To One Model:</u> • Many User Level Threads are mapped to one Kernel level Thread. • Does Not support parallel processing of threads in Multicore systems. • If 1 thread makes a blocking call, then entire process will stop	Many to One Model 
2. <u>One To One Model:</u> • Each user thread maps to a unique kernel thread. • Multiple threads can run on multiple processors. • Blocking in one thread does not affect others. • Overhead is higher because each user thread requires a corresponding kernel thread.	One to One Model 
3. <u>Many to Many model:</u> • Multiple user threads map to multiple kernel threads. • If one thread blocks, others can continue. • Provides high concurrency and is the most efficient model.	Many to Many Model 

Benefits of Multithreading

- Responsiveness (program does not freeze): A multithreaded application increases responsiveness to the user.
- Resource sharing (threads share memory): Resources like code and data are shared between threads

- Economy (less costly than process creation): Thread switching is very efficient
- Scalability (runs faster on multi-core CPUs): New item can be added to existing one.
- Context switching: Context Switching is used for storing the context or state of a process so that it can be reloaded when required.

Thread Libraries:

A thread library provides the programmer with an API (set of functions) to create, manage, and synchronize threads.

Three primary thread libraries:

1. POSIX Threads (Pthreads):

- A standard for thread programming defined by IEEE POSIX.
- Mostly used in UNIX, Linux, macOS.

Command: `pthread_create(&tid, NULL, function, NULL);`

How it works:

Pthreads provides functions for:

- Creating a thread → `pthread_create()`
- Terminating a thread → `pthread_exit()`
- Waiting for a thread → `pthread_join()`
- Synchronization → mutexes, condition variables
- Setting thread attributes

Features:

- Portable across UNIX-like systems
- Can support user-level or kernel-level thread.
- Very flexible and powerful.

2. Win32 Threads:

- Thread library used in the Windows OS family (NT, Windows 10, etc.)
- Command : `CreateThread(NULL, 0, function, NULL, 0, &tid);`

How it works:

Win32 API provides functions like:

- `CreateThread()` – create a thread
- `ExitThread()` – terminate
- `WaitForSingleObject()` – wait for thread
- Synchronization: Mutex, Semaphore, Events

Characteristics:

- Completely kernel-level threads
- Integrates tightly with the Windows subsystem
- Designed for C/C++ and .NET environments.

3. Java Threads

- Built into the Java language using the java.lang.Thread class.
- Managed by the JVM, not the OS directly.

How it works:

Two ways to create a thread:

- Extend Thread class
- Implement Runnable interface

Core Methods:

- start() → begin thread execution
- run() → code executed by the thread
- sleep() → pause
- join() → wait for a thread
- Synchronization → synchronized keyword, locks, monitors.
-

Threading Issues

These are problems that arise when working with threads:

1. Fork() and Exec() Issues:

- Should new threads be copied?
- Should only the calling thread survive?

2. Signal Handling:

Two types:

- Synchronous signals → delivered to calling thread
- Asynchronous signals → delivered to specific or all threads

3. Thread Cancellation:

Stopping a thread:

- Asynchronous cancellation

- Deferred cancellation

4. Thread-Local Storage (TLS):

Each thread needs its own private data (not shared).

5. Scheduler Activations:

Used for many-to-many models to balance user threads and kernel threads.

BCS303